

Análisis de capacidad

Carga de documentos:

Estas pruebas de carga fueron diseñadas para evaluar los dos flujos más importantes de la aplicación. El primer flujo abarca la etapa de *chunking* y *embedding* del documento, hasta el almacenamiento de los vectores en la base de datos vectorial. El *chunking* consiste en dividir un documento en fragmentos más pequeños (chunks) para facilitar su procesamiento y búsqueda, mientras que el *embedding* transforma cada fragmento en una representación numérica (vector) que captura su significado semántico. El segundo flujo, por su parte, corresponde a la interacción del usuario con el chat en función de los documentos previamente cargados.

Para poder cumplir con este fin se idearon dos escenarios:

Escenario 1 (Carga del documento):

Este escenario no posee un contexto previo, busca probar la crear credenciales y demás recursos para poder lograr la carga y almacenamiento del documento.

- Se crea un usuario, mediante el uso del endpoint de usuarios (El cual se utiliza para crear un usuario)
- Se hace login mediante el endpoint de usuarios/login, donde este me retorna el id del usuario y el token de acceso correspondiente (Importante para poder consumir los endpoints de los microservicios posteriores)
- Se crea un chat para el usuario correspondiente, mediante el uso del endpoint /chats
- Se carga un archivo a este chat mediante el uso del endpoint de chat.

Este Flujo fue creado en Jmeter, y el archivo correspondiente se llama CSV_Data_Set_prompt.jmx dentro de la carpeta Jmeter en el repositorio relacionado al proyecto (<https://github.com/leytoncasta/app-web-notebook-lm>), el cual se ve de la siguiente manera:

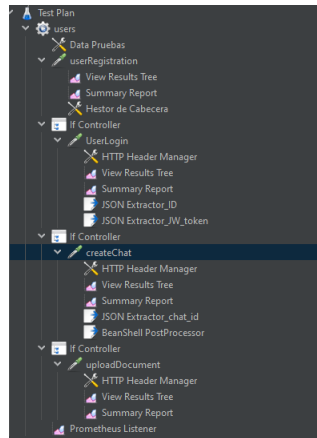


Figura 1. Pruebas Jmeter escenario 1.

Escenario 2 (Preguntas sobre el documento):

Este escenario utiliza los datos que se crearon anteriormente para la carga de documento, guardando el chat id y el token bearer que utilizaremos para la autenticación de la aplicación. Dado esto esta prueba seguirá los siguientes pasos.

- Se cargan los datos antes guardados por el escenario 1.
- Se hace una misma pregunta “¿Qué dice el texto?” para todas las threads.

Todo esto al final se ve de la siguiente manera:

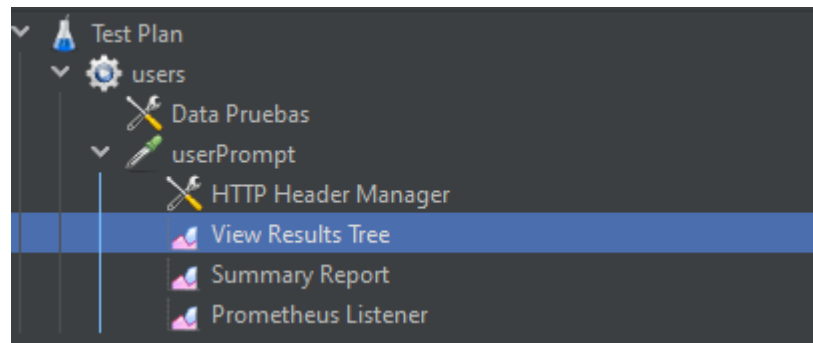


Figura 2. Pruebas Jmeter escenario 2.

Resultados:

Para las pruebas de carga se usó Jmeter, Prometeus y Grafana para poder observar los resultados que implican calcular tiempos de respuesta y la cantidad de solicitudes que terminaron con un status 200 o no.

Se tiene que aclarar que tal como se mencionó en el documento perteneciente a la entrega de la planeación del análisis de carga, se enviaron **150 hilos (threads)** distribuidos en un período de **120 segundos**, utilizando un único ciclo (*loop*). Por lo tanto, todos los resultados que se presentan a continuación corresponden a esa configuración de prueba.

Por ende, se calcularon las siguientes métricas utilizando estas plataformas de observabilidad:

Throughput:

Este indicador se define como el número de solicitudes con código de estado 200 procesadas por minuto. Para calcularlo, se configuró en Grafana una tasa (rate) de respuestas exitosas a partir de la métrica denominada **throughput_total**. Como se muestra en la *Figura 3*, se utilizó el listener de Prometheus, y en la *Figura 4* se presenta la configuración correspondiente en Grafana.

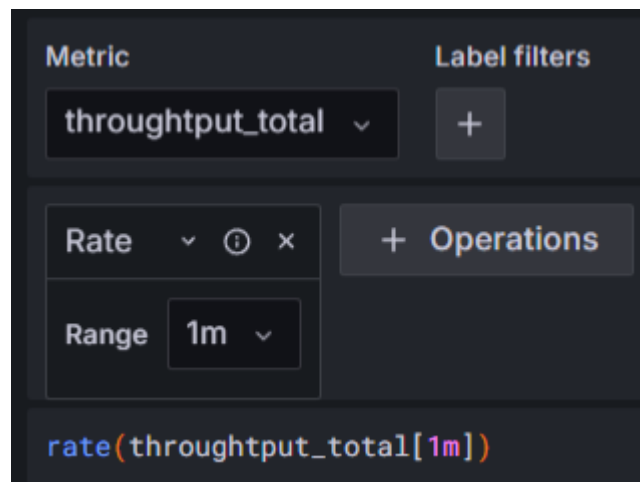


Figura 3. Configuración de Grafana.

Concurrent Users:

Para estimar los usuarios concurrentes en la aplicación, se calcula el tiempo de respuesta promedio en segundos y el throughput en solicitudes por segundo, usando `rate()` con una ventana de 1 minuto. Aunque la ventana es de un minuto, el resultado de `rate()` representa una tasa por segundo, ya que Prometheus divide el cambio total en ese intervalo entre su duración en segundos. Multiplicando el throughput por el tiempo de respuesta se obtiene la cantidad de solicitudes simultáneas, es decir, cuántas están activas en un momento dado.

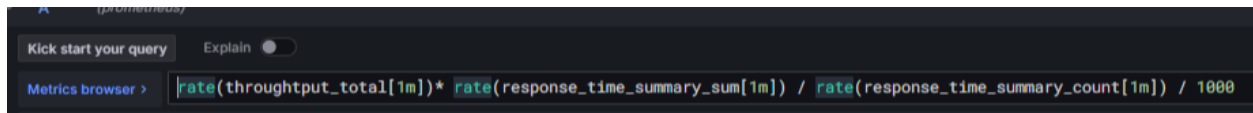


Figura 4. Configuración Grafana Concurrent users.

Concurrent Users Total:

Es la estimación de la suma de usuarios concurrentes por segundo durante ese minuto, obtenida al agregar los valores de concurrencia calculados previamente para todos los endpoints existentes.

throughput_total	default help string	label	COUNTER		samples	SuccessTotal
response_time_summary	default help string	label	SUMMARY		samples	ResponseTime

Figura 5. Configuración Jmeter con el listener de Prometheus.

Response Times:

Esta métrica abarca los tiempos de promedio por minuto de cada tipo de request. Básicamente se trata de los tiempos que se han demorado las solicitudes, dividido la cantidad de solicitudes, como se puede ver en la *Figura 5*, y la configuración de esta métrica en Jmeter se puede ver en la *Figura 3* como **response_time_summary**.

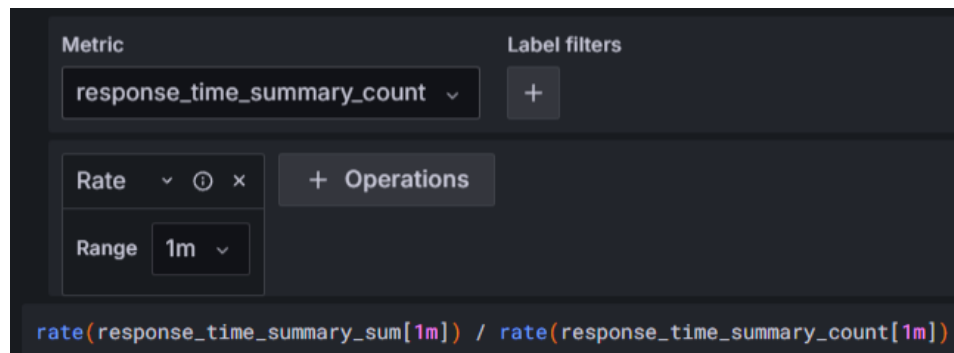


Figura 6. Configuración de Response Times en Grafana.

Successful Request:

Esta métrica es simplemente un conteo del número de solicitudes que devolvieron un estado 200. Como se trata de un contador en Prometheus, este no disminuye, ya que va acumulando valores. Gracias a esta métrica podemos ver cuántas solicitudes fueron procesadas y en qué momento la aplicación se detuvo o no pudo continuar.

Resultados Escenario 1:

En cuanto a los resultados, se puede observar claramente en la *Figura 6* el pico en el que comienzan a enviarse solicitudes con estados diferentes a 200, alrededor de las 00:10:30. Como se muestra en los tiempos de respuesta, en ese momento aún no hay un impacto en el desempeño de la aplicación. Por lo tanto, podríamos decir que, considerando el número

de respuestas exitosas —cercano a los 74 usuarios y haciendo upload del documento a 70—, a partir de ese punto el sistema empieza a verse comprometido. En el minuto siguiente, los tiempos de respuesta aumentan de forma drástica, y aunque el sistema logra responder algunas solicitudes adicionales, lo hace con tiempos de respuesta muy elevados, y en algunos casos ni siquiera logra dar respuesta.

A partir del análisis realizado, se observa que la aplicación fue capaz de manejar hasta 70 solicitudes dirigidas a la sección de documentos sin mayores inconvenientes. No obstante, al analizar la métrica de usuarios concurrentes, se evidencia que hacia las 00:10:30 —justo antes de que comenzara la degradación del sistema— había un promedio de 4.79 usuarios concurrentes por segundo realizando solicitudes a los distintos endpoints. Esta métrica puede apreciarse en las gráficas correspondientes al throughput total. A partir de ese punto, como se mencionó anteriormente, el sistema comienza a degradarse progresivamente.

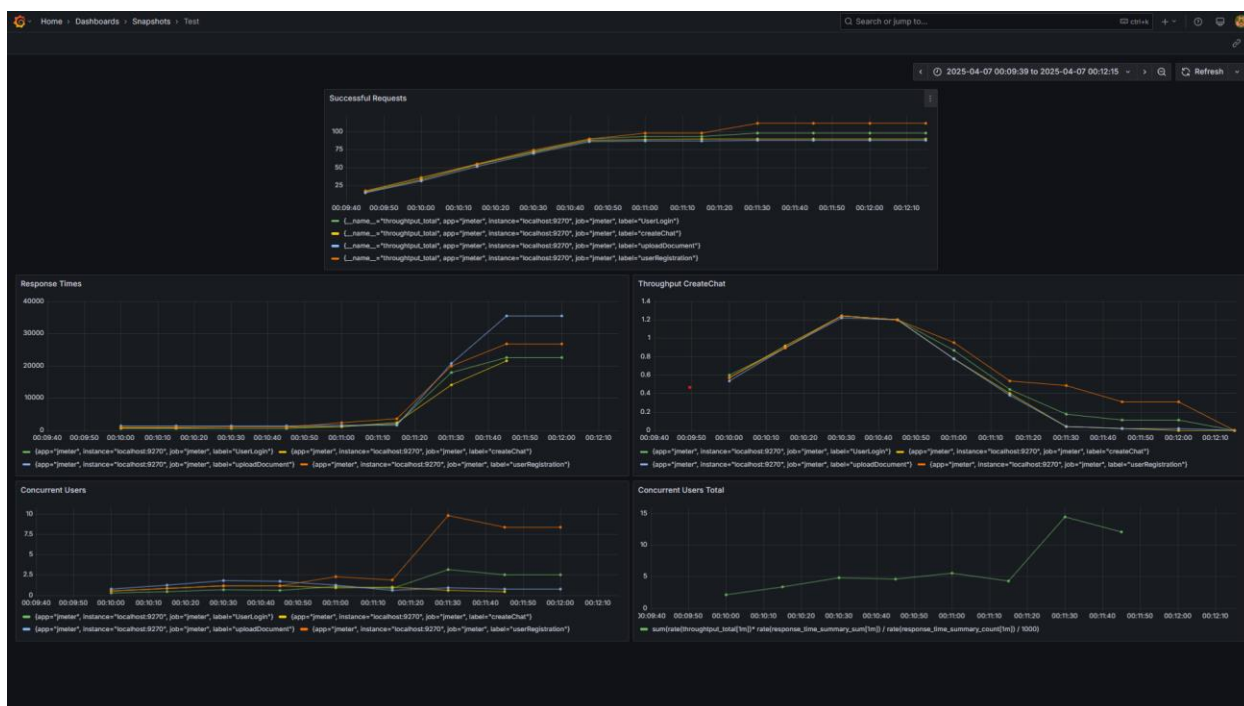


Figura 7. Resultados escenario 1.

Por otro lado, cuando vamos a Cloud Monitoring del worker vemos que la maquina no se ve alterada en cuanto a la CPU, ni a memoria lo que hace pensar que la administración de los contenedores Docker es el verdadero problema. Por ende, se recomienda revisar las configuraciones de estos dado que, pareciera la maquina sigue teniendo la capacidad de soportar más carga.



Figura 8. Uso de la CPU de las pruebas de carga Escenario 1 del worker .

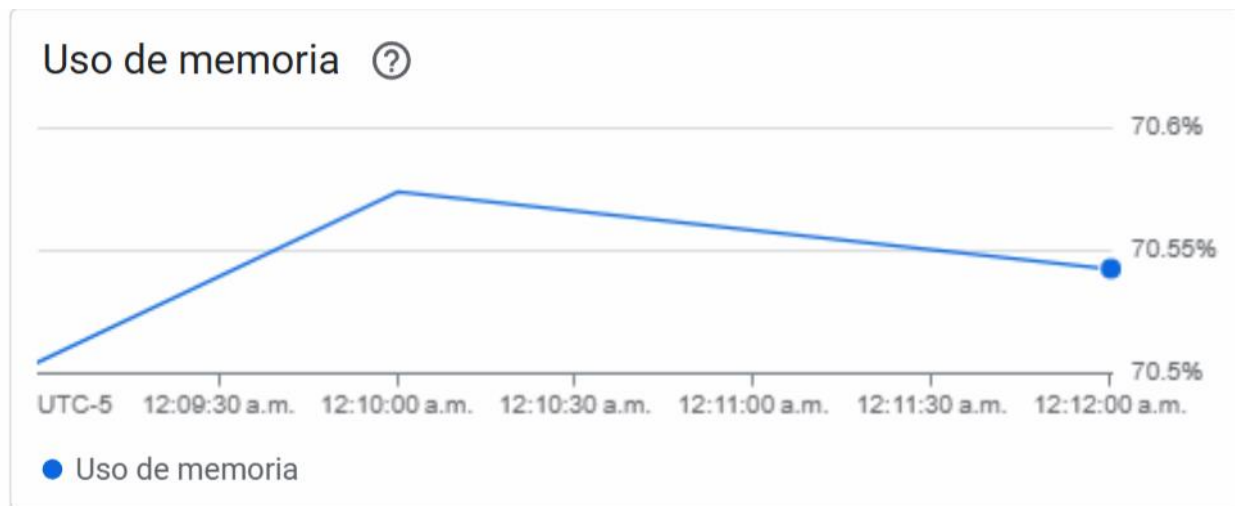


Figura 9. Uso de la Memoria de las pruebas de carga Escenario 1 del worker.

Por ende, el punto de fallo no está realmente aquí, pero cuando vemos el uso de CPU del back, tenemos que está se sobresaturo probablemente porque es la encargada de dejar los documentos al File Storage, por ende, el cuello de botella está en el backend.

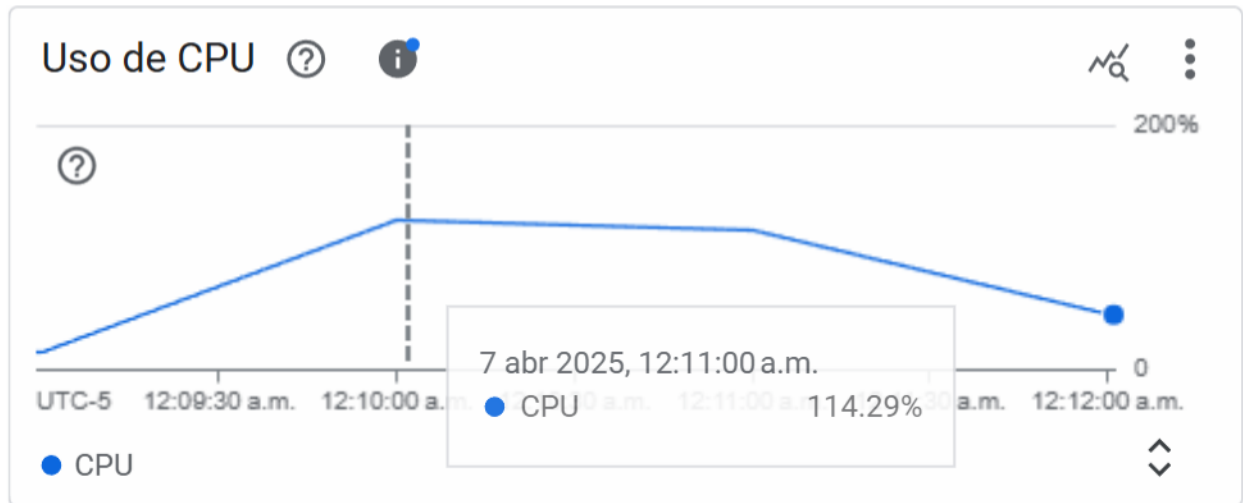


Figura 10. Uso de la CPU de la máquina del backend para el escenario 1.

Resultados Escenario 2:

Con respecto al escenario 2, al observar el fragmento de pruebas presentado, se evidencia que el sistema es apenas capaz de soportar 1 usuario concurrente por segundo, como se aprecia en el pico previo a la degradación de los tiempos de respuesta. Si bien no se registra una caída abrupta como en el escenario anterior, el incremento en la latencia confirma que el sistema entra en un estado de degradación. Por lo tanto, se concluye que el límite para mantener un funcionamiento óptimo es de 1 usuario concurrente.



Figura 11. Resultados escenario 2.

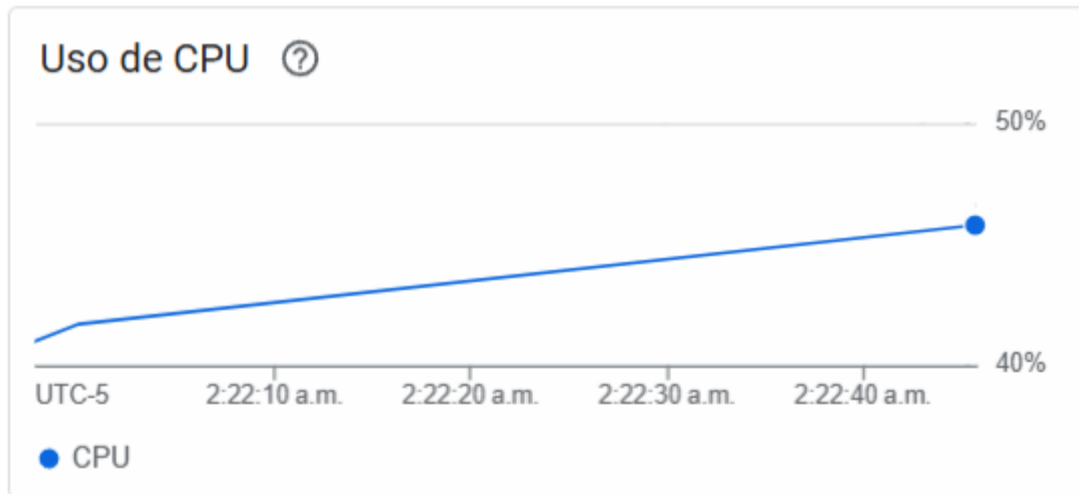


Figura 12. Resultados utilización de CPU worker escenario 2.

Con respecto a la utilización de recursos de la máquina del Worker. Podemos ver que la máquina sigue subutilizada, por ende, el punto de demora podría estar dado por las configuraciones del contenedor o por tiempos de cola o similar el API de Gemini. Por ende, es posible que concurrentemente este escenario soporte un aproximado de 4 usuarios concurrentes antes de degradarse.